MGPEB

Gerenciamento de pouso na missão Aurora Siger

Relatório técnico - Atividade Integradora da Fase 2

FIAP - Módulo de Gerenciamento de Pouso e Estabilização de Base (MGPEB)

Grupo 2

Integrante	RM	Participação no trabalho
Lorenzo Mendes Rocha	RM575361	Cenário, protótipo, regras de decisão e modelo matemático
Davi Coninck Cassaro	RM572772	Testes e documentação das estruturas de dados
Eduardo Backes Klauck	RM575889	Evolução da computação e sistemas embarcados
Gabriel Gonzales	RM574746	ESG e governança da base
Guilherme Cardoso Bremenkamp	RM574229	Integração, revisão e organização da entrega

Introdução

Na aproximação a Marte, os módulos da missão Aurora Siger precisam aguardar uma autorização de pouso. A ordem de atendimento e as condições de cada tentativa afetam a chegada de recursos essenciais à base, como energia, suprimentos e equipamentos de saúde.

Neste trabalho, representamos essa operação por meio de um protótipo em Python. Uma fila organiza os módulos que aguardam atendimento, enquanto listas separam os resultados das decisões. As regras consideram combustível, integridade dos sensores, atmosfera e disponibilidade da área. Também usamos buscas e ordenação para consultar os registros e reorganizar a fila.

Além do programa, analisamos o consumo de combustível durante a espera por uma função linear. A discussão sobre computação embarcada ajuda a explicar as escolhas de estruturas e algoritmos. Ao final, relacionamos essas escolhas a questões de segurança, uso de recursos e participação nas decisões da base.

O relatório apresenta as seis etapas da atividade e inclui um anexo com os registros usados no exemplo de execução. O código completo e as instruções para reproduzir os resultados estão no repositório do grupo.

AURORA SIGER

POUSO E ESTABILIZAÇÃO DE BASE

1. Organização dos módulos e da fila

Escolhemos seis tipos de módulos: Saúde, Segurança, Energia, Comunicação, Suprimentos e Transporte. A escolha acompanha necessidades de uma base em formação: atendimento médico, proteção, abastecimento de energia, comunicação, alimentação e água, além da movimentação de pessoas e cargas.

Cada registro reúne os atributos abaixo. No programa, o nome indica o tipo do módulo; prioridade e criticidade são informações separadas. Um módulo de Saúde, por exemplo, não recebe prioridade automaticamente apenas pelo nome.

Atributo	Valores usados na geração dos dados
Nome do módulo	Saúde, Segurança, Energia, Comunicação, Suprimentos ou Transporte
Prioridade de pouso	Inteiro de 1 a 5; 1 representa a maior urgência
Nível de combustível	Inteiro de 0 a 100%
Massa	Inteiro de 10 a 20 toneladas
Criticidade da carga	Baixa, Média ou Alta
Chegada estimada à órbita	12/10/2026, entre 06:00 e 17:59
Integridade dos sensores	Operacional, Falha parcial ou Falha total

As classes `Modulo` e `CondicoesAmbiente`, em `Modulos.py`, organizam esses dados. A atmosfera assume os estados Estável, Tempestade ou Ventos fortes. A área de pouso pode estar Livre, Ocupada ou Obstruída.

Em `main.py`, todos os módulos começam na fila `modulos_fila_de_pouso`. A inclusão ocorre com `append()` e a retirada com `popleft()`, usando `deque`. Assim, o primeiro registro da fila é o primeiro a ser atendido. Depois da tentativa, cada módulo entra em uma das listas: `modulos_pousados`, `modulos_em_espera` ou `modulos_em_alerta`.

A lista `modulos_definidos` mantém os registros cadastrados, e `condicoes_externas_ambiente` guarda os ambientes na ordem das tentativas. As decisões são registradas em `historico_de_decisoes`, usado como pilha.

A execução padrão sorteia seis registros, podendo repetir tipos de módulos. No exemplo fixo, usamos um de cada tipo e reorganizamos a fila por prioridade antes do atendimento. A retirada continua FIFO na nova ordem. Cada registro é processado uma vez; os módulos adiados não voltam automaticamente à fila.

2. Regras de autorização de pouso

Para representar as regras com portas lógicas, definimos quatro entradas: **F** indica falha total dos sensores; **E**, combustível abaixo de 10%; **A**, atmosfera estável; e **P**, área livre. Cada entrada vale 1 quando a condição indicada é verdadeira e 0 quando é falsa.

A decisão segue a ordem das condições em `main.py`:

Ordem	Condição	Resultado
1	F	Bloqueado
2	NOT F AND E	Autorizado (emergência)
3	NOT F AND NOT E AND (NOT A OR NOT P)	Adiado
4	NOT F AND NOT E AND A AND P	Autorizado

O circuito reúne a autorização normal e a de emergência em uma única saída. Suas expressões são: **Bloqueado = F**; **Autorizado = (NOT F) AND (E OR (A AND P))**; **Adiado = (NOT F) AND (NOT E) AND ((NOT A) OR (NOT P))**. Os parênteses deixam explícita a ordem das operações.

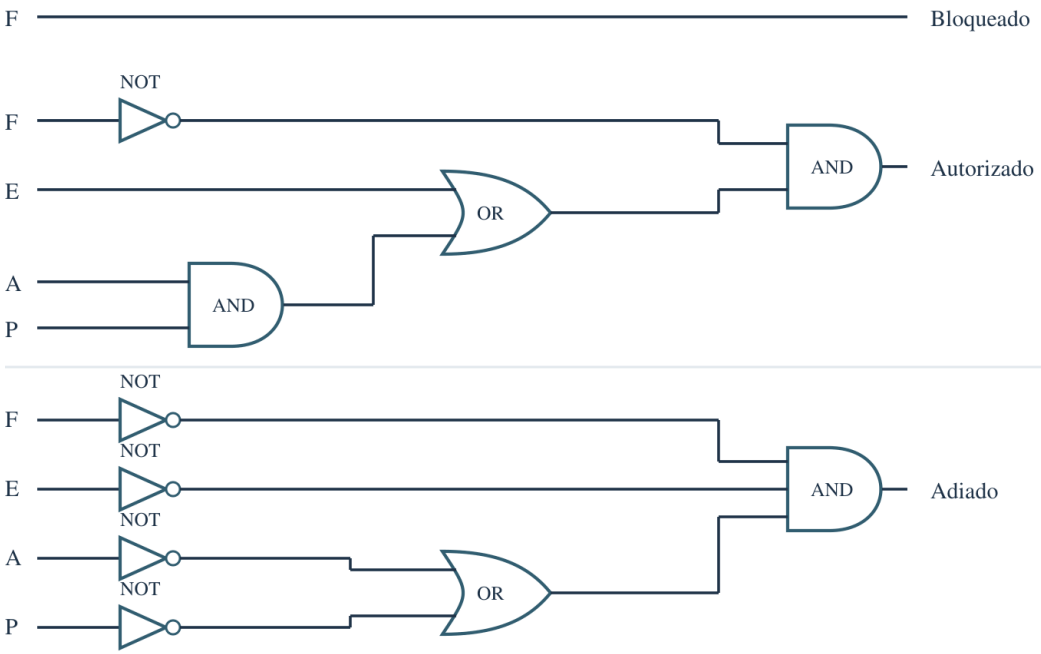


Figura 1 - Circuito das decisões, com portas AND, OR e NOT. Fonte: elaboração do grupo.

A falha total é verificada primeiro. Por isso, um módulo com sensores em falha total fica bloqueado mesmo com combustível abaixo de 10%. Na ausência dessa falha, o combustível baixo produz autorização de emergência. Com 10% ou mais, o pouso depende de atmosfera estável e área livre.

Mantivemos essa regra de emergência do protótipo, mas ela tem um limite importante: pode autorizar com atmosfera desfavorável ou área ocupada. A autorização é o resultado da lógica do exercício; não demonstra que a descida seria viável nessas condições. A emergência também não altera a fila nem escolhe uma área alternativa. Falha parcial dos sensores, por sua vez, não produz bloqueio automático.

As saídas do circuito são exclusivas: cada combinação gera apenas uma decisão. A comparação com o programa foi verificada em 135 combinações de estados dos sensores, combustível, atmosfera e área, incluindo os limites de 9%, 10% e 11%.

3. Implementação e execução do protótipo

O código foi dividido em quatro arquivos. `Modulos.py` define os registros; `Dados.py` gera módulos e ambientes; `Consultas.py` contém as buscas e a ordenação; e `main.py` processa a fila e apresenta os resultados. O programa usa funções, condicionais, laços e recursos da biblioteca padrão para manipular os dados.

Em `Consultas.py`, as buscas percorrem a coleção para encontrar o módulo com menor combustível, o de maior prioridade ou todos os registros de determinada criticidade. A maior prioridade corresponde ao menor número. Nas buscas por um único registro, uma coleção vazia retorna `None`; na busca por criticidade, retorna `[]`.

A função `ordena_fila()` usa Bubble Sort. Ela compara registros vizinhos e troca suas posições quando o valor anterior é maior. Os critérios são prioridade, combustível, massa e chegada, em ordem crescente. Como valores iguais não são trocados, os empates mantêm a ordem de entrada. A função devolve um novo `deque`, preservando a fila recebida.

Para executar na raiz do projeto, usamos Python 3.10 ou superior:

```
python3 main.py
python3 -m exemplos.exemplo_deterministico
python3 -m unittest discover -s tests -v
```

O primeiro comando executa a geração aleatória. O segundo usa os seis registros fixos apresentados no anexo: chama as três buscas e as quatro ordenações, aplica a fila por prioridade e passa os ambientes para `decide_pouso()`. Dessa forma, é possível repetir os mesmos resultados.

O exemplo termina com Energia, Comunicação e Suprimentos na lista de pousados; Transporte e Segurança em espera; e Saúde em alerta. Energia recebe autorização de emergência, enquanto Saúde é bloqueada pela falha total dos sensores.

Foram executados 21 testes: os 15 cenários da validação original e seis verificações de integração. Eles incluem decisões de pouso, FIFO, LIFO, buscas, ordenação, coleções vazias e empates. Um desses testes compara as expressões booleanas com o processamento em 135 combinações.

O histórico é exibido com `pop()`, da decisão mais recente para a mais antiga. Essa operação remove os registros, deixando a pilha vazia. O exemplo guarda uma cópia antes da exibição para permitir acompanhar a sequência. O programa não mantém um registro permanente das decisões.

4. Consumo de combustível durante a espera

Representamos o consumo durante a espera em órbita por uma função linear. Para o exercício, adotamos uma taxa constante para as correções de trajetória e orientação:

$$C(t) = C_0 - k \times t$$

Parâmetro	Significado	Unidade
C(t)	Combustível restante após t horas	% da capacidade do tanque
C_0	Combustível no início da espera	%
k	Taxa constante de consumo	pontos percentuais por hora
t	Tempo de espera	horas
C_{min}	Reserva de referência	10%

Com k=5 pontos percentuais por hora e reserva de 10%, o tempo disponível até atingir essa reserva é:

$$t_{limite} = (C_0 - C_{min}) / k$$

São cinco pontos percentuais do tanque por hora, não 5% do saldo restante. A tabela apresenta três exemplos.

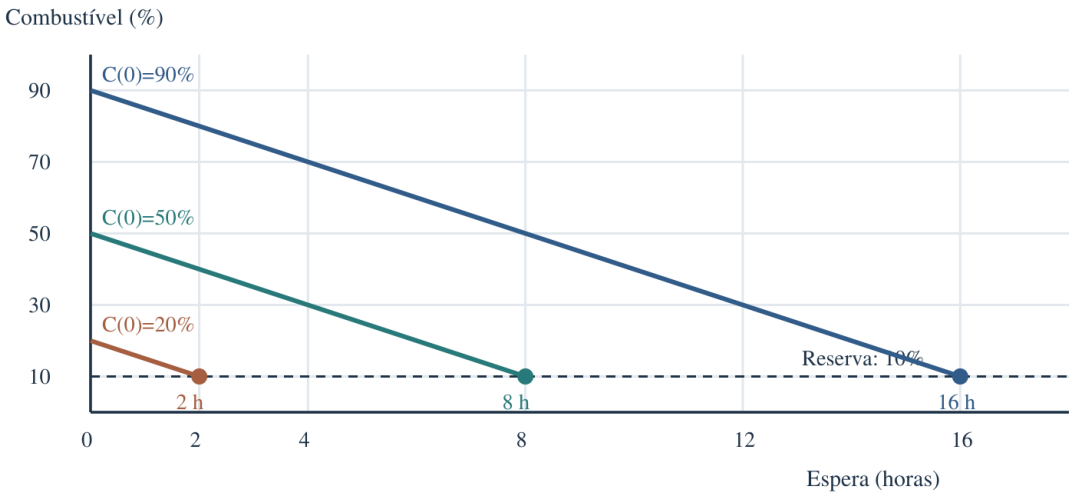


Figura 2 - Redução linear do combustível até a reserva de 10%. Fonte: elaboração do grupo.

Caso	C_0	Substituição na fórmula	t_{limite}
A	20%	(20 - 10) / 5	2 h
B	50%	(50 - 10) / 5	8 h
C	90%	(90 - 10) / 5	16 h

As retas são decrescentes, com inclinação -5, e terminam na reserva de 10%. Aumentar o combustível inicial prolonga o tempo disponível; aumentar a taxa de consumo reduz esse tempo.

Para k>0, a função mantém combustível não negativo entre 0 e C_0/k. A espera com reserva vai de 0 a t_{limite}, se $C_0 >= C_{min}$. Abaixo de 10%, a reserva já foi violada; com k=0, a fórmula de tempo limite não se aplica.

O modelo sugere limitar a espera e priorizar quem tem menos tempo disponível. Essa aplicação não foi implementada: o programa usa combustível fixo e prioridade informada ou sorteada. Os 20% são apenas um caso do gráfico; o limite de emergência do código continua abaixo de 10%.

5. Computação e restrições de uma missão espacial

A evolução dos computadores permitiu passar de equipamentos de grande porte, baseados em válvulas, para circuitos com transistores, circuitos integrados e microprocessadores. A redução de tamanho e o processamento dedicado abriram espaço para sistemas embarcados. Em uma missão espacial, porém, capacidade de processamento e miniaturização precisam ser avaliadas junto com energia, memória e confiabilidade.

No MGPEB, usamos uma amostra de seis registros. Nesse tamanho, a busca linear e o Bubble Sort tornam o funcionamento fácil de acompanhar. A busca tem custo O(n), enquanto a ordenação apresenta custo $O(n^2)$. Se a quantidade de módulos crescer, as comparações do Bubble Sort passam a pesar mais; uma ordenação O(n log n) seria uma alternativa a estudar.

Restrição	Relação com o projeto	Limitação da escolha
Processamento	Buscas lineares e Bubble Sort permitem acompanhar cada comparação	O custo cresce com a quantidade de registros
Memória	Listas guardam estados; deque organiza a fila; uma lista representa a pilha	Mais registros exigem mais memória; o histórico não é permanente
Energia	O processamento percorre a fila uma vez e usa condicionais simples	Não medimos consumo elétrico nem desempenho em um dispositivo
Radiação e confiabilidade	Testes ajudam a repetir e conferir as regras do programa	Não há redundância de hardware nem proteção contra radiação

As operações de inclusão e retirada nas extremidades de `deque` têm custo aproximadamente constante. Para a pilha, `append()` e `pop()` operam no final da lista [2]. Essas estruturas atendem ao exercício sem exigir uma implementação própria de fila ou pilha.

A radiação espacial pode afetar circuitos e provocar efeitos de evento único, entre outros danos descritos pela NASA [3]. Esses problemas exigem medidas de hardware e avaliação do ambiente da missão; não são resolvidos pela escolha de uma estrutura de dados em Python.

Também distinguimos as condições de órbita e superfície. A superfície de Marte possui atmosfera rarefeita, não vácuo. O protótipo não calcula temperatura, dissipação de calor ou consumo de energia. Sua contribuição é mostrar como escolhas simples de programação organizam a operação; a adaptação a um sistema de voo exigiria outras verificações.

6. ESG e governança da base Aurora

As decisões de pouso afetam a formação da base e o uso de recursos. Por isso, a análise do MGPEB não se limita à ordem da fila. Consideramos os impactos ambientais das operações, as necessidades das pessoas e a responsabilidade por definir e revisar as regras.

Na dimensão ambiental, a escolha da área deveria considerar estabilidade do terreno, dispersão de poeira e proteção de regiões de interesse científico. A base também precisaria organizar o uso de energia, a contenção de resíduos e os limites de produção e extração de recursos locais, como gelo e regolito. Propomos acompanhar essas atividades antes de ampliá-las, evitando tratar os recursos como ilimitados.

Na dimensão social, Saúde e Energia representam cargas relevantes para atendimento e suporte à base. A prioridade dessas cargas deveria fazer parte de critérios discutidos com operadores e moradores. No programa, entretanto, o nome do módulo não determina sua prioridade: esse vínculo precisaria ser definido nas regras de cadastro.

Para a governança, propomos um comitê técnico com participação das pessoas afetadas para revisar prioridades e limiares. Um operador responsável avaliaria exceções e registraria suas justificativas. O quadro reúne situações do próprio exemplo e o que ainda precisaria ser tratado.

Situação	Comportamento observado	Questão para a governança
Energia com 9%, tempestade e área ocupada	Autorização de emergência	Rever o conflito entre urgência e condições da área
Segurança com 90% e área ocupada	Pouso adiado	Definir quem reavalia a próxima tentativa
Saúde com falha total dos sensores	Pouso bloqueado, mesmo com 5%	Estabelecer procedimento para falha de instrumentação
Comunicação com falha parcial	Pouso autorizado em ambiente favorável	Avaliar quais falhas parciais permitem uma descida
Histórico exibido com pop()	Pilha esvaziada	Definir retenção das decisões para posterior consulta

A transparência exige permitir que outra pessoa entenda por que uma decisão ocorreu. Um registro mais completo deveria guardar as entradas, a regra aplicada, o resultado, o motivo e a versão do programa. Hoje, o histórico contém apenas nome e decisão e é consumido na exibição.

Essas medidas de participação, revisão e retenção são propostas para a base. O código não implementa confirmação humana, auditoria em banco ou desvio para áreas alternativas. O exemplo permite discutir esses limites a partir de situações concretas, em vez de apresentar os princípios ESG de forma isolada da operação.

Anexo - Listas, fila e pilha no exemplo

Os registros abaixo pertencem ao exemplo `exemplos/exemplo_deterministico.py`. A massa está em toneladas e todos os horários se referem a 12/10/2026. A ordem da tabela é a ordem de cadastro na lista `modulos_definidos`.

Módulo	Prioridade	Comb.	Massa	Carga	Chegada	Sensores
Transporte	4	50%	20	Baixa	11:00	Operacional
Comunicação	3	10%	12	Média	08:00	Falha parcial
Saúde	1	5%	10	Alta	06:00	Falha total
Energia	2	9%	15	Alta	07:00	Falha parcial
Segurança	5	90%	18	Alta	12:00	Operacional
Suprimentos	3	11%	14	Média	09:00	Operacional

Lista e consultas. A inclusão usa `append()`. Menor combustível e maior prioridade retornam Saúde. A busca por criticidade Alta retorna Saúde, Energia e Segurança na ordem de cadastro. Pela massa, a ordem obtida é Saúde, Comunicação, Suprimentos, Energia, Segurança e Transporte.

Fila. Aplicando a ordenação por prioridade, a retirada com `popleft()` segue a sequência abaixo. Comunicação permanece antes de Suprimentos no empate de prioridade 3. Depois das seis retiradas, a fila está vazia.

Módulo retirado	Atmosfera / área	Resultado / lista de destino
Saúde	Estável / Livre	Bloqueado / alerta
Energia	Tempestade / Ocupada	Autorizado (emergência) / pousados
Comunicação	Estável / Livre	Autorizado / pousados
Suprimentos	Estável / Livre	Autorizado / pousados
Transporte	Ventos fortes / Livre	Adiado / espera
Segurança	Estável / Ocupada	Adiado / espera

Pilha. As decisões são inseridas com `append()` na mesma ordem das retiradas. Ao exibir com `pop()`, a ordem se inverte: Segurança, Transporte, Suprimentos, Comunicação, Energia e Saúde. A pilha fica vazia. A descrição completa das operações está em `doc/anexo_estruturas.md`.

Referências

[1] FIAP. A Aurora Ajusta a Trajetória na Aproximação a Marte. Capítulo 1, pp. 14-17. Material da disciplina.

[2] PYTHON SOFTWARE FOUNDATION. collections - deque; Data Structures. Acesso em: 13 out. 2026.

[3] NASA. What is Space Radiation?. Acesso em: 13 out. 2026.

[4] GRUPO 2. Código e documentação do MGPEB. Contribuições registradas nos PRs #1, #6, #7 e #8.